

Hyperparameters

Settings, values, and their justification

PAUT Explainable-AI Digital Twin — weld defect inspection from Total Focusing Method (TFM) images.

Why every hyperparameter has the value it does, how it was chosen, and where the value comes from. Values are the production settings in `configs/*.yaml` and the classifier code. The formulas these feed are independently verified against the code by `scripts/verify_math.py` (all checks pass — see the end of this doc).

Three sources of a value are used throughout:

- **Architecture-fixed** — set by the supervisor-mandated design (IWSCN reference paper); not free to change.
- **Theory/standard** — a standard recipe from the literature (cited).
- **Empirically chosen** — selected on the validation split / by cross-validation (never on test).

1. Wavelet-Scattering branch (`configs/model.yaml` → `model.scattering`)

Param	Value	Why this value	How chosen
J (scale)	2	Each order-1 coefficient downsamples by 2^J . $J=2 \rightarrow 64 \times 64$ feature maps from 256×256 , keeping enough spatial resolution to localise small defects; $J=3$ would drop to 32×32 and blur tiny porosity.	Architecture-fixed (IWSCN); confirmed adequate in the type-classifier search (<code>scripts/tune_type_classifier.py</code> tried $J=3$, no gain).
L (orientations)	8	8 wavelet orientations (every 22.5°) capture directional weld texture (slag is elongated, porosity round).	Theory (standard Kymatio default; Bruna & Mallat).
order	2	Order-2 scattering captures interactions between scales — needed to separate texture classes; order-1 alone is too weak.	Architecture-fixed (IWSCN).

Coefficient count (verified): $1 + J \cdot L + L^2 \cdot J(J-1)/2 = 1 + 16 + 64 = 81$. This is the number of scattering channels, checked against the code in `scripts/model_spec.py`.

2. Network architecture (`configs/model.yaml` → `model`)

Param	Value	Why this value	How chosen
base	32	Encoder width 32→64→128→256, bottleneck 512 → 8.8 M params. Large enough to learn weld texture, small enough to train on a CPU/free GPU with 1050 images without heavy overfitting.	Empirically (standard U-Net width that fits memory).
seg_classes	3	background / porosity / slag — per-pixel segmentation needs a background class.	Architecture-fixed (decision §3).
n_classes	2	porosity vs slag image-level type.	Architecture-fixed (decision §2).
dropout	0.1	Light regularisation; higher values hurt the already-small defect signal.	Empirically.
num_groups	8	GroupNorm groups. GroupNorm (not BatchNorm) because batch size is tiny (4) — BatchNorm statistics would be noisy. 8 groups is the standard GN default.	Theory (Wu & He, GroupNorm).
size_head	false	Defect size is measured geometrically from the mask (regionprops), which is exact, instead of regressed (which would need size labels we don't have).	Architecture-fixed (decision §11).

3. Loss weights (configs/model.yaml → loss)

$L = w_{\text{seg}} \cdot L_{\text{seg}} + w_{\text{cls}} \cdot \text{CE} + w_{\text{boundary}} \cdot \text{boundary}$, with $L_{\text{seg}} = \text{dice} \cdot \text{Dice} + \text{focal} \cdot \text{FocalCE} + \text{tversky} \cdot \text{FocalTversky}$.

Param	Value	Why this value
w_seg	0.6	Segmentation (detection) is the primary task → largest weight.
w_cls	0.3	Classification is secondary (type is finally decided by the scattering classifier).
w_boundary	0.1	A small nudge for crisp defect edges; too high distracts from region overlap.

Param	Value	Why this value
tversky_alpha	0.3	Weight on false positives .
tversky_beta	0.7	Weight on false negatives . $\beta > \alpha$ (0.7 > 0.3) makes *missed* defect pixels hurt more than false alarms — correct for thin/faint defects that fail by being missed.
tversky_gamma	1.0	Focal-Tversky exponent. 1.0 = plain Tversky; left at 1.0 because $\beta > \alpha$ already handles the imbalance and higher γ destabilised training.
focal_gamma	2.0	Focal-CE focusing parameter; 2.0 is the standard value (Lin et al., Focal Loss) — down-weights easy background pixels.
include_background_in_dice	false	Critical: averaging Dice over foreground only. With background included the model scored ~0.66 Dice by predicting *all background* and abandoning the defect (the collapse bug). Excluding background removes that free credit.

Why these weights and not others: the split 0.6 / 0.3 / 0.1 follows the task priority (detect > classify > refine edges); the Tversky α/β follow the standard imbalance recipe (Salehi et al.); the include_background=false choice was forced by the observed collapse.

4. Training schedule (configs/model.yaml → train)

Param	Value	Why this value	How chosen
epochs	150	Enough for the cosine schedule to converge; early-stopping ends runs sooner if val Dice plateaus.	Empirically.
batch_size	4	Largest batch that fits in CPU/free-GPU memory at 256×256; small batch is *why* we use GroupNorm.	Hardware-constrained.
lr	0.001	Standard Adam starting LR; with warmup + cosine it anneals to ~0. Higher diverged, lower was slow.	Empirically (standard default).
weight_decay	1e-4	Light L2 regularisation, standard for Adam-family on small data.	Theory/standard.

Param	Value	Why this value	How chosen
warmup_epochs	5	Avoids large early steps destabilising the fresh weights before cosine decay.	Standard recipe.
scheduler	cosine	Smooth anneal → better final minima than step decay on small data.	Standard recipe.
grad_clip	1.0	Caps gradient norm so the focal/Tversky terms can't spike the update.	Standard recipe.
early_stop_patience	25	Stop if val foreground-Dice hasn't improved in 25 epochs — guards against overfitting the tiny set.	Empirically.
class_weighting	true	Inverse-frequency weighting on the classification CE. The dataset is now 525/525 balanced, so it is effectively neutral — kept as a safeguard.	Safeguard.

Augmentation (configs/model.yaml → augment, train split only): h/v flips & 90° rotations (p=0.5) are valid because defect appearance is orientation-agnostic; gain (0.8–1.2), speckle ($\sigma=0.05$) and attenuation (0.7) simulate real PAUT acquisition variation so the model is robust to scanner/gain differences.

5. Type classifier (src/models/type_classifier.py)

Param	Value	Why this value	How chosen
pipeline	log1p → StandardScaler → MLP(64)	log1p is the standard *log-scattering* renormalisation (Mallat/Bruno); StandardScaler centres features; a tiny 64-unit MLP is the "small classifier" of the IWSCN recipe.	5-fold CV on train (tune_type_classifier.py).
MLP hidden	64	Big enough to separate the two texture classes, small enough not to overfit 735 images.	CV-selected over LogReg / SVM / MLP.
alpha (L2)	1e-2	MLP weight decay; CV-tuned for best balanced accuracy.	CV.
result	0.88 balanced (vs 0.80 plain LogReg)	—	CV 0.87 ≈ test 0.88 → generalises.

6. Severity & digital-twin (configs/characterize.yaml, configs/twin.yaml)

All values here are ****tunable placeholders, NOT validated acceptance criteria**** — they must be calibrated to ISO 5817 / ASME BPVC with a domain expert before any real pass/fail use.

Param	Value	Status
severity.type_weight porosity/slag	1.0 / 1.2	Placeholder (slag weighted higher as generally more critical).
severity.size_ref_px	200	Placeholder area→severity reference.
twin.capacity	5.0	Placeholder severity "budget" in health = $1 - \min(1, \Sigma \text{severity} / \text{capacity})$.
twin.review_health	0.7	Placeholder: health below this → REVIEW.
cutoffs moderate / critical	0.5 / 1.0	Placeholder severity bands.

7. Math == code verification

scripts/verify_math.py recomputes each formula in independent numpy and asserts it matches the production code (tolerance 1e-6). On a hand-computable 2×2 / 3-class example:

Quantity	By hand / numpy	Code	Match
Foreground Dice loss	0.407895	0.407895	✓
Focal-Tversky loss ($\alpha 0.3, \beta 0.7$)	0.414948	0.414948	✓
Focal-CE loss ($\gamma 2$)	0.079805	0.079805	✓
Health index ($\Sigma \text{sev } 1.2, \text{ cap } 5$)	0.760	0.760	✓
Health index ($\Sigma \text{sev } 4.5, \text{ cap } 5$)	0.100	0.100	✓

Reproduce: `py -3.14 -m scripts.verify_math` (exits 0 = all match).